From DSL to Native in One Process: JIT-Compiling Exo Kernels via xDSL and llvmlite

Abstract

Exo is a user-schedulable language for writing optimized hardware kernels, but it only emits C code, requiring a separate compile-link-load cycle to call kernels from Python. We present a JIT compilation path that lowers Exo's intermediate representation through xDSL (a Python-native MLIR framework) to LLVM IR, then compiles to native code via llvmlite's MCJIT engine, all within a single Python process. We evaluate on matrix multiplication (256×256, `float32`), applying Exo's scheduling transforms (loop reordering, tiling, unrolling) and measuring their impact. The best kernel achieves 7 GFLOPS on scalar AArch64 code: 11× over the naive baseline, 110× over CPython. The compiler is ∼1,250 lines of Python.

Introduction

Writing fast numerical code in Python typically means writing C. Implement the kernel, compile to a shared library, load via `ctypes`, hope you got the pointer types right. Change anything (reorder a loop, adjust a tile size) and you repeat the cycle. Tolerable for production code, painful for exploration.

Exo [ikarashi2022exo] is a Python-embedded DSL for hardware kernels. You write a loop nest and transform it through semantics-preserving *scheduling primitives*: reorder loops, tile, unroll. If the schedule typechecks, the kernel computes the same result. But Exo only generates C. You still need a C compiler, a build system, and an FFI bridge.

Can we skip C entirely? Instead of `Exo → C → clang → .so → ctypes`, we want `Exo → native code → call it`, in one Python process.

We use xDSL [fehr2025xdsl] (Python-native MLIR) to lower Exo's IR to the LLVM dialect, then llvmlite [llvmlite] (from Numba) to JIT-compile via MCJIT. Result: a callable function pointer, usable with NumPy arrays through `ctypes`. No files on disk, no external compiler.

This is a feasibility study. The full pipeline is ∼1,250 lines of Python. We evaluate on matrix multiplication and show the workflow (write, schedule, run, measure, iterate) works in a single Python session. No claim to compete with BLAS.

Background

Exo

Exo [ikarashi2022exo] is a language for *exocompilation*: instead of hiding hardware-specific optimizations inside the compiler, it exposes them as user-level scheduling operations. Write a loop nest (the *algorithm*), transform it through rewrites (the *schedule*). Transforms:

- `fission`: split a loop body into two loops.
- `reorder_loops`: swap two nested loops.
- `divide_loop`: replace a loop with outer + inner (tiling).
- `unroll_loop`: fully unroll a constant-bound loop.

Each transform preserves semantics. Exo currently emits C only.

xDSL

xDSL [fehr2025xdsl] reimplements MLIR's core infrastructure (SSA-based IRs, dialects, regions, pattern rewriting) in pure Python. Same `memref`, `llvm`, `builtin` dialect operations as upstream MLIR, no C++ LLVM build. Compiler passes, lowering pipelines, IR manipulation, all in Python.

llvmlite

llvmlite [llvmlite] provides a Python API for constructing LLVM IR and JIT-compiling via MCJIT. Developed for Numba, usable independently. Construct an `ir.Module`, parse into MCJIT, finalize, retrieve a function pointer by name. Wrap in `ctypes.CFUNCTYPE`, call from Python.

System Design

Pipeline Overview

Four stages, each a Python function call. No subprocesses, no disk I/O:

```
Exo @proc  ->  LoopIR           [Exo]
   -> MLIR (memref+llvm)        [xDSL]
   -> MLIR (llvm only)          [xDSL passes]
   -> LLVM IR -> MCJIT -> fn*   [llvmlite]
   -> ctypes callable           [Python]
```

IR Generation

The IR generator (633 lines) walks Exo's LoopIR, emits xDSL operations. Expressions become `llvm.constant`, `llvm.fadd`, `memref.load`. Statements map to structured patterns:

- *For loops* become three basic blocks (header, body, exit) with an induction variable as block argument.
- *Reductions* (`C[i,j] += ...`) lower to load–add–store.
- *Allocations* become `llvm.call @malloc` (heap) or `llvm.alloca` (stack).
- *Subprocedure calls* trigger recursive lowering of the callee.

Type mapping is direct: `f32` → `f32`, `i32` → `i32`, tensors → `memref<shape x element_type>`. Exo's analysis passes run before lowering.

Memory Lowering

Standard MLIR lowers `memref` through *descriptors* (structs with base pointer, offset, sizes, strides per dimension). General but adds overhead: every load/store extracts struct fields and computes a strided index.

Exo tensors have static shapes and row-major layout. We bypass descriptors entirely, lowering `memref.load`/`memref.store` to direct pointer arithmetic:

$$\texttt{ptr} = \texttt{base} + \left(\sum_d \texttt{index}_d \times \texttt{stride}_d \right) \times \texttt{sizeof(elem)}$$

where $\texttt{stride}_d = \prod_{d'>d} \texttt{dim}_{d'}$ from the static shape. This pass (290 lines) rewrites all `memref` ops into `llvm.getelementptr` + `llvm.load`/`llvm.store` before types are erased to opaque pointers.

JIT Compilation and Calling Convention

The final stage (82 lines) translates the xDSL LLVM-dialect module to llvmlite IR and invokes MCJIT. Forward-declare functions, emit bodies by walking xDSL ops and calling llvmlite builder methods, parse the textual LLVM IR, verify, compile. The execution engine is kept alive by storing a reference on the ctypes function object.

Memref arguments become opaque pointers at the LLVM level. For matmul: `void kernel(float* C, float* A, float* B)`. From Python, NumPy arrays pass via `.ctypes.data`; function type is `CFUNCTYPE(None, c_void_p, c_void_p, c_void_p)`.

Evaluation

256×256 f32 matmul ($C = A \times B$), $2 \times 256^3 \approx 33.6$M flops. Apple Silicon (AArch64), scalar instructions only, no NEON, no AMX. Medians of 20 runs, 3 warmup.

Kernel Versions

Naive baseline in Exo, optimized variants via scheduling transforms:

```
@proc
def v0_naive(C: f32[256, 256] @ DRAM,
             A: f32[256, 256] @ DRAM,
             B: f32[256, 256] @ DRAM):
    for i in seq(0, 256):
        for j in seq(0, 256):
            C[i, j] = 0.0
            for k in seq(0, 256):
                C[i, j] += A[i, k] * B[k, j]
```

Scheduling `v0_naive`:

```
# v1: separate init from accumulate, reorder j<->k
v1 = fission(v0, v0.find("for k in _:_").before(),
             n_lifts=2)
v1 = reorder_loops(v1, "j k")

# v3: v1 + tile j by 4, unroll inner j
v3 = divide_loop(v1, "j #1", 4, ["jo","ji"],
                 perfect=True)
v3 = unroll_loop(v3, "ji")
```

Table 1: Performance on 256×256 `float32` matmul (Apple Silicon, scalar AArch64).

Kernel	ms	GFLOPS	vs v0	vs Py
python (ikj)	524.6	0.06	0.1x	1x
v0_naive	52.8	0.64	1.0x	10x
v1_reorder	6.1	5.51	8.7x	86x
v2_k_tile	6.0	5.56	8.7x	87x
v3_unroll	4.9	6.79	10.7x	106x
v4_2d_tile	9.8	3.42	5.4x	53x

A fifth variant (`v4_2d_tile`) is a manually written $32 \times 32 \times 32$ blocked kernel. The baseline is a pure Python implementation using nested lists (same `ikj` loop order as `v1`).

Results

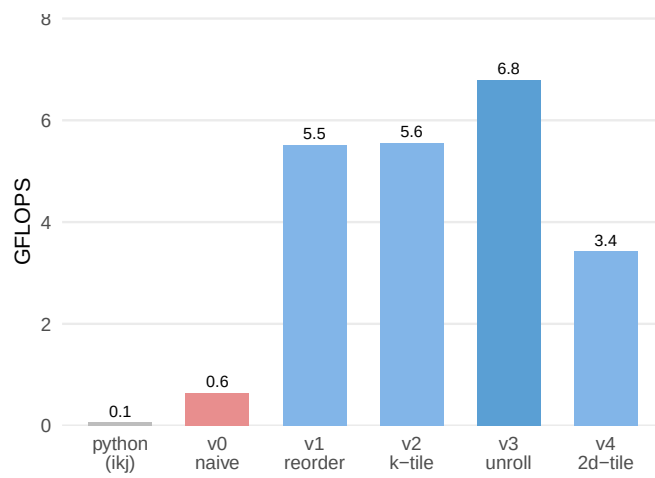

Figure 1: Throughput (GFLOPS) by kernel version. All Exo kernels are scalar. The Python baseline uses pure list-of-lists arithmetic.

Analysis

Cache reordering dominates. The `ijk` → `ikj` transform (v0 → v1) gives 8.7×. In `ijk`, the inner loop sweeps `k`, accessing `B[k, j]` stride-256, a different cache line every iteration. In `ikj`, the inner loop sweeps `j`, both `C[i, j]` and `B[k, j]` are stride-1. Single most impactful optimization.

k-tiling is marginal at this size. Tiling `k` by 32 (v2) shows negligible improvement over v1. A row of A is 1~KB at 256×256; the hardware prefetcher handles it. Would matter at 2048^2.

j-unrolling adds 1.3×. Dividing inner `j` by 4 and unrolling (v3) exposes four independent accumulations, enabling ILP. The CPU pipelines multiply-adds across four lanes without data dependencies.

Manual 2D tiling underperforms. The 32^3 tiled kernel (v4) is 1.8× slower than v3. Complex index expressions (`32*io + ii`) generate more instructions; LLVM optimizes them less aggressively than the simpler loop nests

from Exo's scheduling. More tiling is not always better.

Python to v3 is 110×. The full journey: (1) eliminating interpreter overhead (10×), (2) fixing cache access patterns (8.7× more), (3) exposing ILP (1.3× more). Same result, same multiply-add count throughout.

Discussion

What works. Define a kernel, schedule, JIT-compile, benchmark. One Python script, no Makefile. Scheduling guarantees correctness: you change *how* the kernel computes, not *what*. The whole pipeline is ∼1,250 lines of Python, small enough to read and modify in an afternoon.

Backend comparison. Figure 2 compares end-to-end time for a 16×16 matmul across three compilation paths: (1) Exo's default C path (emit C, invoke clang, load `.so`), (2) xDSL lowering with external MLIR tools, (3) our JIT path via llvmlite. The JIT path runs in 0.47~ms, 4.5× faster than the C path (2.1~ms) and 6.5× faster than the MLIR toolchain (3.1~ms). The difference is subprocess and disk I/O overhead; the JIT path stays in-process.

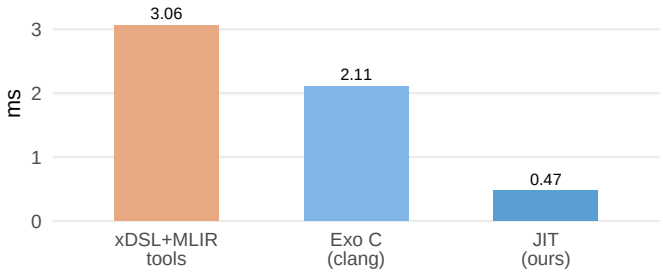

Figure 2: End-to-end time for 16×16 matmul across compilation backends. JIT (ours) avoids subprocess and disk I/O overhead.

Code quality. The generated LLVM IR is clean: flat loops with GEP/load/fmul/fadd/store sequences. MCJIT runs LLVM's optimization passes (register allocation, instruction scheduling). No memref descriptors means no struct unpacking, no hidden indirections. For v3, the unrolled inner loop contains four independent multiply-accumulate chains that LLVM pipelines successfully.

Limitations. Purely scalar. NumPy (Apple Accelerate) hits ∼1,100 GFLOPS on the same hardware, 160× faster, via NEON SIMD and AMX. The pipeline supports vector intrinsics (58 AVX2-style ops are already lowered) but not evaluated here; focus is the scheduling workflow, not peak perf. Exo also requires static array sizes, limiting generality.

Viability. Good for prototyping kernel designs, precisely where the C compilation overhead hurts most. Production deployment (e.g., custom convolution kernels) needs SIMD and larger sizes; the infrastructure exists, evaluation is future work.

Extensibility. Adding an Exo operation: one pattern match in the IR generator, one llvmlite builder call. The xDSL IR is inspectable: print the MLIR at any stage, diff against expectations. Unlike C++ MLIR, no rebuild to debug lowering.

Comparison to Numba. Numba [numba] JIT-compiles Python via llvmlite through type inference, no scheduling transforms. We trade general Python compatibility for explicit control over loop structure and memory access patterns via Exo's scheduling language.

Related Work

Halide [ragan2013halide] pioneered the algorithm/schedule separation for image processing pipelines. Exo builds on this but externalizes hardware knowledge to user-defined libraries rather than embedding it in the compiler. TVM [chen2018tvm] extends the Halide model with learning-based auto-tuning for deep learning workloads. JAX [jax] provides trace-based JIT compilation via XLA with a NumPy-like interface but without user-directed scheduling control. Our work is complementary: a JIT backend for a scheduling language (Exo), using a Python-native compiler stack (xDSL + llvmlite) rather than C++ LLVM.

Conclusion

Exo kernels can be JIT-compiled to native code through ∼1,250 lines of Python, no C step. On 256×256 matmul, loop reordering gives 8.7×, the full schedule reaches 11×, same gains you'd expect from the same transforms in C. The stack: (1) Exo for kernel specification and scheduling, (2) xDSL for IR manipulation, (3) llvmlite for code generation. Runs entirely in Python, practical for quick iteration on kernel designs.

References

[ikarashi2022exo] Y. Ikarashi, G. L. Bernstein, A. Reinking, H. Genc, and J. Ragan-Kelley. Exo: Exocompilation for productive programming of hardware accelerators. In *PLDI*, 2022.

[fehr2025xdsl] M. Fehr et al. xDSL: A common compiler ecosystem for languages that target dedicated accelerators. In *CGO*, 2025.

[llvmlite] Numba Development Team. llvmlite: A lightweight LLVM Python binding. https://github.com/numba/llvmlite.

[numba] S. K. Lam, A. Pitrou, and S. Seibert. Numba: A LLVM-based Python JIT compiler. In *LLVM-HPC Workshop*, 2015.

[ragan2013halide] J. Ragan-Kelley et al. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *PLDI*, 2013.

[chen2018tvm] T. Chen et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *OSDI*, 2018.

[jax] J. Bradbury et al. JAX: Composable transformations of Python+NumPy programs. https://github.com/jax-ml/jax.